By: Shoumik, Moneba, Judy, Ekam & Abrar

Group Project: Team AB
FreshRoute System Design Document

Team Members:

Name	Email	Student ID
Moneba Hanifi	moneba@my.yorku.ca	219721810
Shoumik Ahmed	shoumik@my.yorku.ca	219881598
Judy El Ghargomi	jelg21@my.yorku.ca	220227237
Ekambir Momi	ekam727@my.yorku.ca	219424407
Abrar Jawad Tarafder	abrart03@my.yorku.ca	219453497

3311 Section M Term W
Class Instructor: Ilir Dema
v2 Updated March 23, 2026

Table of Contents

Cover Page 1
Table Of Contents 2

Chapter 1: Document Overview

Introduction 3

Chapter 2: System Overview and Design

Architecture Diagram 4
MVC Diagram 5
Component Design CRC Cards 6
Assumption and Future Work 10
Sprint Specifications 11
References 12

Chapter 1: Document Overview

INTRODUCTION

This document describes the system design for **FreshRoute**, a web-based grocery platform focused on organic and ethically sourced food products. The purpose of this document is to present the overall system structure, the technologies used in its implementation, and the responsibilities of the major components.

FreshRoute follows a **MERN stack architecture**, comprising a React frontend, a Node.js and Express backend, and a MongoDB database accessed via Mongoose. Communication between the frontend and backend occurs through **RESTful APIs using HTTP and JSON**, with Cross-Origin Resource Sharing (CORS) enabled during development.

The backend follows a **Model-View-Controller (MVC)** design pattern. Express routers define API endpoints, controllers handle incoming requests and application logic, and models define the MongoDB schemas and manage database interactions. The backend configuration is handled in **app.js**, while **server.js** serves as the application entry point, establishing the database connection and starting the HTTP server.

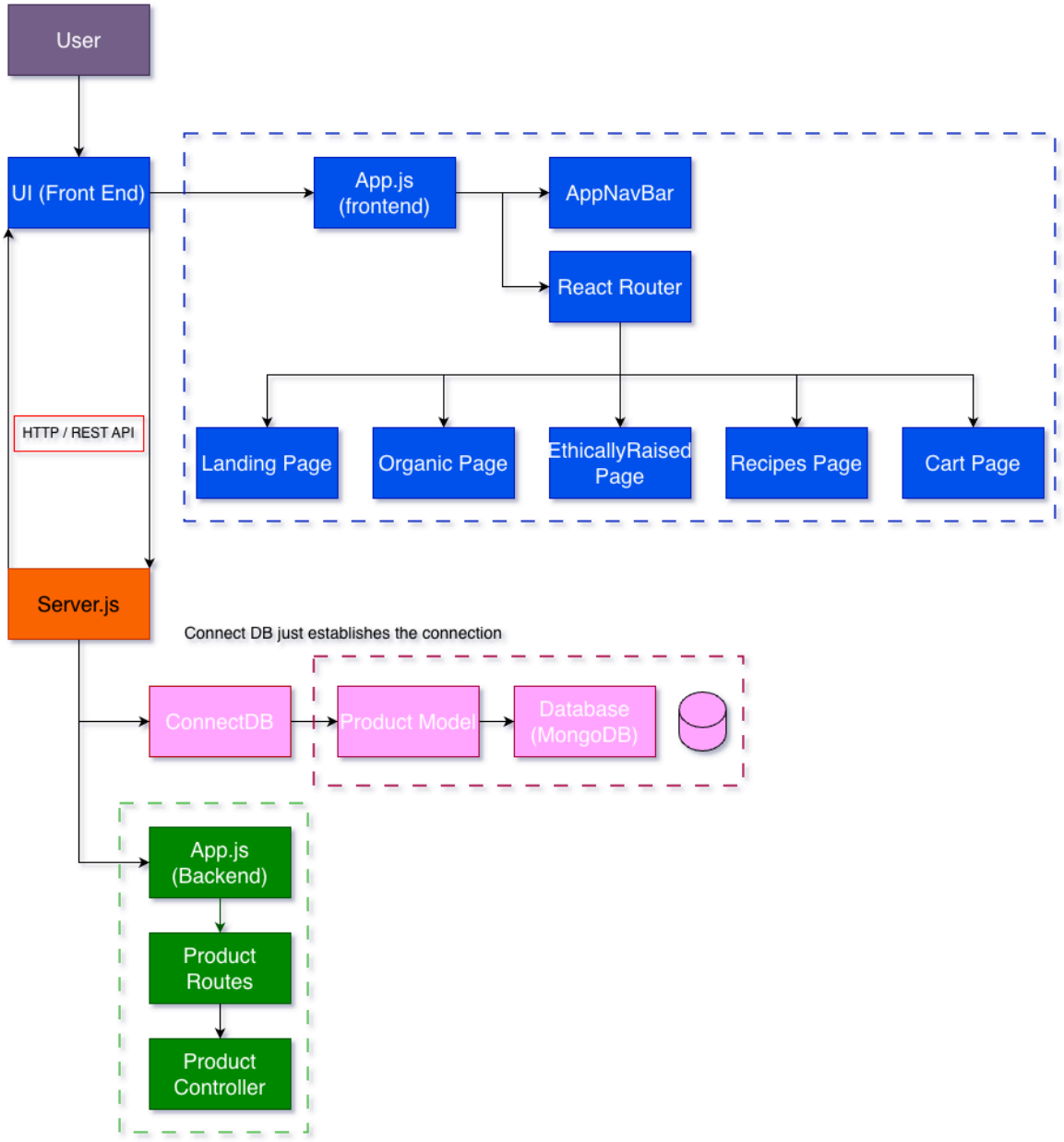
The frontend is implemented as a component-based React single-page application using Vite and styled using **Bootstrap and React-Bootstrap components**. The root component (**App.jsx**) manages **client-side routing based on URL paths** and loads page components, including **Landing, Organic, Ethically Raised, Recipes, and Cart**. These components share common interface elements such as the navigation bar (**AppNavbar**).

Data currently flows from the frontend to the backend through API endpoints such as **/api/products**, which return JSON responses. The frontend currently manages state locally within components without a global state management system.

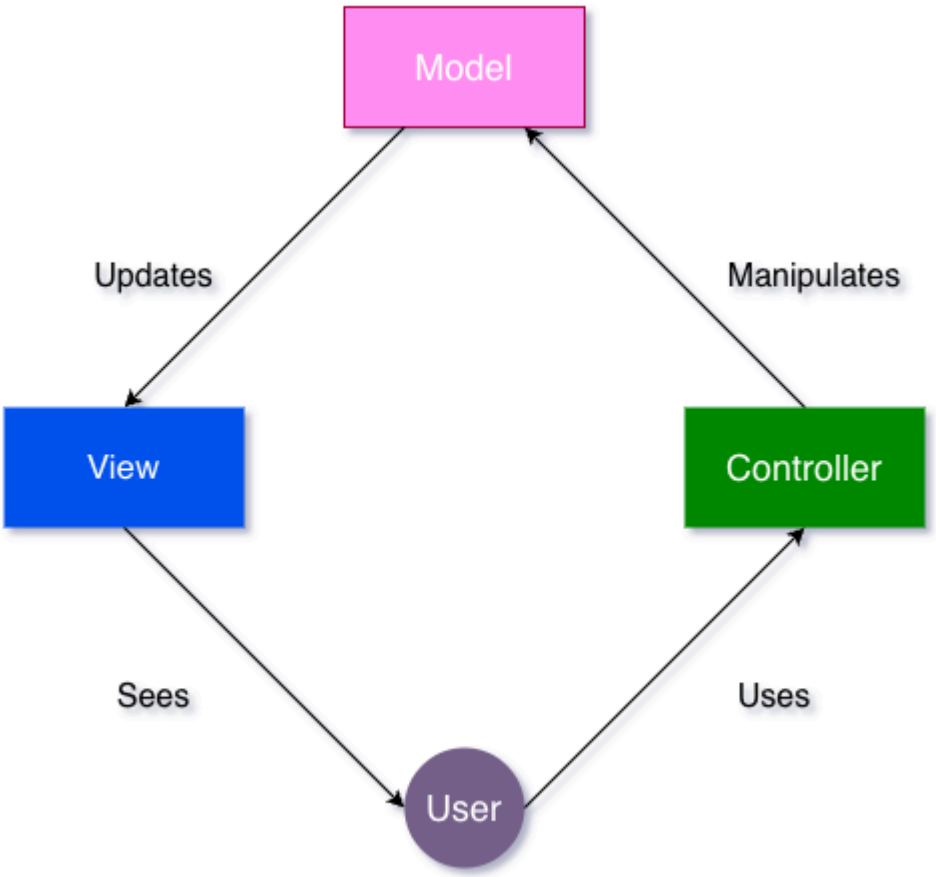
During development, the backend server runs on **port 4000** while the frontend development server runs on **port 5173**. Environment variables, such as the MongoDB connection string, are configured in a **.env file**.

Chapter 2: System Overview

Architecture Diagram



MVC Diagram



COMPONENT DESIGN

CRC CARDS

Frontend Components: React Components

App (frontend)

Frontend

Parent Class: React component
Subclasses: Landing, Organic, EthicallyRaised, Recipes, Cart

Responsibilities:

- Controls which page is shown based on the URL.
- Acts as the main container for the frontend.
- Loads the appropriate page component.

Collaborators:

- Landing, Organic, EthicallyRaised, Recipes, Cart

AppNavbar

Frontend

Parent Class: React component
Subclasses: None

Responsibilities:

- Displays navigation links to main sections.
- Provides search/location input.
- Shows the cart icon and user greeting.

Collaborators:

- All pages (shared navbar)

Landing

Frontend

Parent Class: React component
Subclasses: None

Responsibilities:

- Displays the homepage content.
- Introduces the store and its values.
- Provides a button to start shopping.

Collaborators:

- AppNavbar

Organic

Frontend

Parent Class: React component
Subclasses: None

Responsibilities:

- Shows organic produce in a grid layout.
- Allows users to search and filter products.
- Displays product details in a card.

Collaborators:
- AppNavbar

EthicallyRaised

Frontend

Parent Class: React component
Subclasses: None

Responsibilities:

- Shows ethically raised produce in a grid layout.
- Allows users to search and filter products.
- Displays product details in a modal.

Collaborators:
- AppNavbar

Recipes

Frontend

Parent Class: React component
Subclasses: None

Responsibilities:

- Displays a list of recipes.
- Allows users to search recipes and view details.
- Lets users add recipe ingredients to the cart and automatically adjusts quantity in terms of user set serving size.

Collaborators:
- AppNavbar

Cart

Frontend

Parent Class: React component
Subclasses: None

Responsibilities:

- Displays items added to the cart.
- Provides the structure for cart functionality.

Collaborators:
- AppNavbar

Backend Components: Node / Express / MongoDB Components

server.js

Backend

Parent Class: None
Subclasses: None

Responsibilities:

- Starts the server and loads environment settings.
- Connects the application to the database.
- Stops the server if the database connection fails.

Collaborators:
- connectDB
- app (Express app)

app.js (Express app)

Backend

Parent Class: Express application
Subclasses: None

Responsibilities:

- Configures middleware such as JSON parsing.
- Provides basic routes like health checks.
- Connects API routes to the server.

Collaborators:
- productRoutes

productRoutes

Backend

Parent Class: Express Router
Subclasses: None

Responsibilities:

- Defines API routes for product requests.
- Sends requests to the appropriate controller functions.

Collaborators:
- ProductController

ProductController

Backend

Parent Class: None
Subclasses: None

Responsibilities:

- Handles requests to create and retrieve products.
- Validates data and returns proper HTTP responses.

Collaborators:
- Product model

Product (model)

Backend

Parent Class: Mongoose Model
Subclasses: None

Responsibilities:

- Defines the structure of product data.
- Validates product fields before saving to the database.
- Provides queries used by controllers.

Collaborators:
- MongoDB
- ProductController

connectDB

Backend

Parent Class: None
Subclasses: None

Responsibilities:

- Connects the application to MongoDB.
- Handles database connection errors.

Collaborators:
- Mongoose
- server.js

ASSUMPTION AND FUTURE WORK

The FreshRoute system assumes a development environment that supports **Node.js version 18 or later (Node.js 20+ recommended)** and access to a **MongoDB Community Server**. The backend connects to the database using the `MONGO_URI` environment variable, and the application can run on common operating systems such as **Windows, macOS, or Linux**. During development, the frontend and backend communicate through **HTTP with CORS enabled** to allow requests from the frontend development server. Development typically uses tools such as **VS Code and Git for version control**, though these are not strict requirements.

Error handling and system performance are currently implemented at a basic level. Validation errors return **HTTP 400 responses**, and database connection failures terminate the server after logging an error. The application runs as a **single Node.js process**, which is sufficient for development and course-scale use but may require improvements such as caching or pagination for larger workloads. At the moment, frontend pages primarily display UI components and placeholders, while full API integration for **products, recipes, and cart functionality** will be implemented in later sprints.

Future development may expand the system by introducing **user authentication, improved security measures, and better frontend error feedback**. Additional enhancements may include completing API integration across all pages, improving performance and scalability, and introducing **global state management** to better coordinate data across frontend components.

Sprint Specifications

Sprint 01

- Ship core storefront navigation: Landing → Organic → Ethically Raised → Recipes → Cart via React Router with reusable navbar/search.
- Stand up data layer: MongoDB collections for organic, ethically raised, and recipes
- Express APIs to serve products and details.
- Present rich product info: badges (organic/ethically raised), availability flags, and product detail views powered by seeded/DB data.
- Establish architecture for growth: MVC backend, componentized frontend, Git feature-branch workflow, and initial cart UI with adjustable quantities.

Sprint 02

- Deliver a reliable cart experience: add/view/update/remove items,
- persist the cart across navigation/refresh and capture the checkout address.
- Improve shopping efficiency: product filters/sort on the frontend, backed by organic/ethically raised/recipe data integrations.
- Enable recipe-driven shopping: scale ingredients by servings and add them all to the cart in one action, without duplicates.
- Align frontend–backend data flow: serve product data (including images) from backend to React pages with consistent schemas and routing.

Sprint 03

- TBD in Sprint 03

REFERENCES

- https://www.geeksforgeeks.org/system-design/system-design-tutorial/
- https://guides.visual-paradigm.com/from-use-case-to-mvc-framework-a-guide-object-oriented-system-development/
- https://medium.com/front-end-world/mern-stack-frontend-system-design-ac582521c82f
- https://www.mongodb.com/resources/languages/mern-stack
- https://about.readthedocs.com/?ref=app.readthedocs.org
- https://storybook.js.org/